

JAVA WITH DSA-DAY 2

OOPS CONCEPTS

Class-Object:

```
class printo{
    void disp(){
        System.out.println("this is printscreen");
    }
}

class Main {
    public static void main(String[] args) {
        printo ou=new printo();
        ou.disp();
    }
}
```

Output:

this is printscreen

Encapsulation:

```
class encap{
    private String a;
    public void setname(String a){
        this.a=a;
    }
    public String getname(){
        return a;
    }
}

class Main {
    public static void main(String[] args) {
        encap ep=new encap();
        ep.setname("ABC");
        System.out.println(ep.getname());
    }
}
```

```
}
```

Output:

ABC

Abstraction:

```
abstract class Animal{
```

```
    abstract void display();
```

```
}
```

```
class Dog extends Animal{
```

```
    void display(){
```

```
        System.out.println("Barks");
```

```
    }
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Dog d=new Dog();
```

```
        d.display();
```

```
    }
```

```
}
```

Output:

Barks

Inheritance:

Single level inheritance:

```
class Animal{
```

```
    void display(){
```

```
        System.out.println(" Animals");
```

```
    }
```

```
}
```

```
class Dog extends Animal{
```

```
    void display(){
```

```
        System.out.println("Barks");
```

```
    }
```

```
}
```

```
class Main {  
    public static void main(String[] args) {  
        Dog d=new Dog();  
        d.display();  
    }  
}
```

Output:

Barks

Multi-level inheritance:

```
class Animal{  
    void display(){  
        System.out.println("Animals");  
    }  
}  
  
class Dog extends Animal{  
    void display(){  
        System.out.println("Barks");  
    }  
}  
  
class Cow extends Dog{  
    void display(){  
        System.out.println("Moo");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Cow c=new Cow();  
        c.display();  
    }  
}
```

Output:

Moo

Hierarchical Inheritance:

```
class Animal {  
    void display() {  
        System.out.println("Animal");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Meow");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.display();  
        d.sound();  
        Cat c = new Cat();  
        c.display();  
        c.sound();  
    }  
}
```

Output:

Animal

Bark

Animal

Meow

Super keyword:

Variable:

```
class Animal {  
    String name = "Animal";  
}  
  
class Dog extends Animal {  
    String name = "Dog";  
    void display() {  
        System.out.println(name);  
        System.out.println(super.name);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.display();  
    }  
}
```

Output:

Dog
Animal

Method:

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        super.sound();  
        System.out.println("Dog barks");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output:

Animal sound

Dog barks

Super():

```
class Animal {  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}  
  
class Dog extends Animal {  
    Dog() {  
        super();  
        System.out.println("Dog constructor");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

Output:

Animal constructor

Dog constructor

Polymorphism:

Method Overloading:

```
class MathOperations {
```

```

int add(int a, int b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}
}

class Main {
    public static void main(String[] args) {
        MathOperations m = new MathOperations();
        System.out.println(m.add(2, 3));
        System.out.println(m.add(2, 3, 4));
    }
}

```

Output:

5

9

Method Overriding:

```

class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
    }
}

```

```
        a.sound();  
    }  
}
```

Output:

Dog barks

CONSTRUCTORS

Default Constructor:

```
class Student {  
    Student() {  
        System.out.println("Default Constructor");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
    }  
}
```

Output:

Default Constructor

Parameterized Constructor:

```
class Student {  
    String name;  
    int age;  
    Student(String n, int a) {  
        name = n;  
        age = a;  
    }  
    void display() {  
        System.out.println("Name:" + name);  
        System.out.println("Age:" + age);  
    }  
}
```



```
class Main {  
    public static void main(String[] args) {  
        Student s = new Student("ABC", 21);  
        s.display();  
    }  
}
```

Output:

Name:ABC

Age:21

Copy Constructor:

```
class Student {  
    String name;  
    Student(String name) {  
        this.name = name;  
    }  
    Student(Student s) {  
        this.name = s.name;  
    }  
    void display() {  
        System.out.println(name);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student("Anuk");  
        Student s2 = new Student(s1);  
        s1.display();  
        s2.display();  
    }  
}
```

Output:

Anuk

Anuk

Chain Constructor:

```
class Student {  
    String name;  
    int age;  
  
    Student() {  
        this("ABC", 21);  
    }  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void display() {  
        System.out.println(name + " " + age);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

Output:

ABC 21